From Formal Verification to Agents: A Method-Transfer Audit of "False Completion" in AI Agent Harnesses

liaozhizhi (廖至智)

Contents

Abstract . . . 1
摘要 . . . 2
1 Introduction . . . 2
2 The Source System . . . 4
3 The Target System . . . 5
4 The Transfer . . . 7
5 The Taxonomy . . . 10
6 The Audit Procedure . . . 13
7 Cases . . . 15
8 Measurement . . . 17
9 Threats to Validity . . . 20
10 Conclusion . . . 21
Appendix A — Future work . . . 22
Appendix B — Verification log . . . 22
Appendix C — Sources consulted . . . 24

Title (ZH): 从形式验证到智能体：一次方法迁移审计，及其在"假完成"上的结果

Author: liaozhizhi（廖至智）·liaozhizhi@foxmail.com

Version: 1.0 (preprint). This paper analyses the *transfer* of an audit method from a recomputable system to a non-recomputable one, with the agent domain as the target case rather than the subject.

Status. This document contains **no experiments**. Every worked example is either a property of documented shell/Python semantics (reproducible by the reader in one command, tagged `[SEMANTICS]`), an explicitly labelled single observation (`[OBSERVED]`), or an external figure whose source was located (`[CITED]`). Nothing here is a measurement of prevalence.

Provenance tags.

Tag	Means
`[SEMANTICS]`	a property of the platform, reproducible by the reader in one command
`[OBSERVED]`	a single incident this author witnessed; n = 1, not a rate
`[CITED]`	an external figure whose source was located; Appendix B logs what was checked

Abstract

An agent that reports `Task Completed` has produced a **claim**, not an observation. In 2026 measurements this is not a corner case: 65.4% of one industry evaluation's agent failures ended in confident false completion [CITED], and a separate 2026 study across two benchmarks found that **no LLM-judge configuration it tested** stayed above AUROC 0.65 at separating false successes from real ones [CITED]. The failure is therefore not only common but resistant to the obvious remedy — asking a second model to look — because that remedy is still reading the agent's own account.

Audit methods are usually presented as though they were about their subject matter. They are just as much about their **medium** — and that becomes visible when the medium changes. This paper takes an audit method built for a formal-verification corpus and asks what survives when it is transferred to AI agent harnesses, a setting that differs on four axes: it is stochastic rather than deterministic, its inputs are private rather than public, it has no reference implementation, and its observables are the system's own narration.

We give a transfer analysis instrument by instrument. Three instruments move. **The falsifiability condition** — a property no mutant can kill is a decoration — carries over unchanged, because it is a statement about signals, not about specifications. **Same-origin detection** moves but degrades: in the source system it was a computation (re-derive both artifacts from public bytes and compare); in the target it is only a *search* for a shared coordinate. **Mutation** moves and *strengthens*, because in the target it needs no correct reference — a condition that must fail can be synthesised. **Recomputation is lost entirely**, and we argue this single loss is the load-bearing fact: it is what forces the metric denominator from a workload to a published probe suite, and what makes `INVALID` the modal verdict rather than an edge case.

Applied to the target, the transferred method yields a four-mode taxonomy of false completion generated from one condition (a signal losing its carrier, moment, referent, or range), an audit procedure over evidence origin, execution witness and mutation probing, a four-valued verdict alphabet, and three metrics with probe-suite denominators.

We are explicit that the *mechanism* is not our discovery: that an agent derives its check from the same reading as its solution — so the check inherits the misreading and passes — has been stated in prose by 2026 industry evaluation work [CITED], and that LLM judges are weak at this is now measured [CITED] (no configuration above AUROC 0.65). Our claim is about the transfer: which instruments survive, what they cost, and which conclusions become unsupportable on arrival. We claim mechanism, not prevalence.

摘要

Agent 报告`Task Completed`时，产出的是**声称**而非**观察**。这在2026 年的测量里不是边角情况：一份产业评测中65.4% 的Agent 失败以自信的假完成收场[CITED]；另一项2026 年跨两个基准的系统研究则发现，它测试的**没有任何**LLM 裁判配置在区分真假成功上超过AUROC 0.65 [CITED]。所以这个失败不仅常见，而且对最显而易见的补救——再叫一个模型来看——是免疫的，因为那个补救读的仍然是Agent自己的自述。

审计方法通常被当作是关于其研究对象的。它同样关乎它的**媒介**——而这在媒介改变时才看得见。本文取一套为形式验证语料构建的审计方法，追问当它被迁移到AI Agent Harness 上时，什么能活下来。目标场景在四个轴上不同：它是随机的而非确定的、输入是私有的而非公开的、没有参考实现、且可观测量就是系统自己的自述。

我们逐件仪器给出迁移分析。三件能搬。**可失败条件**——没有变异体能杀死的属性不是检查，是装饰——原样搬过去，因为它说的是信号，不是规约。**同源检测**搬得过去但会退化：在源系统里它是计算（从公开字节重新生成两个产物再比对），在目标系统里它只是对共享坐标的一次搜索。**变异**搬得过去而且**变强**，因为目标里它不需要正确的参照——一个必须失败的条件可以凭空合成。**重算完全丧失**，我们认为这单一损失是承重事实：正是它把度量分母从工作负载逼成了公开探针套件，也正是它让`INVALID`从边缘判词变成常见判词。

施加到目标上，迁移后的方法产出一个由单一条件生成的假完成四模式分类（信号丢失载体、时刻、指称、值域）、一套覆盖来源/见证/变异注入的审计流程、一个四值判词，以及三个带探针分母的度量。

我们明确说明机制不是我们的发现：Agent 从与解法相同的误读中派生校验、于是校验继承误读并通过，这一点2026 年的产业评测已用散文说过 [CITED]；LLM 裁判在这件事上很弱，也已被测量 [CITED]（无配置超过AUROC 0.65）。我们的主张是关于**迁移**的：哪些仪器能活、代价是什么、以及哪些结论一落地就不再成立。我们主张机制，不主张普遍率。

---

1 Introduction

1.1 The question this paper asks

An audit method is usually defended by its results: it found these defects, so trust it on those. That defence is weaker than it looks, because it does not say *why* the method worked, and therefore does not say when it will stop working. The usual way to find out is to try it somewhere else.

This paper does that deliberately. We take an audit framework built for a formal-verification corpus — where the inputs are public, the tool is deterministic, and the audit could re-run the generator — and transfer it to AI agent harnesses, where none of those three things holds. The transfer is the paper's subject. The agent-side findings are its output.

Two things make this more than an exercise.

First, **the failure is the same failure.** In the verification corpus, a formal engine returned `PROVEN` about a property that had been generated from the same specification as the design it was checked against. The proof was valid and the conclusion was empty: it established that two co-generated artifacts agree, not that the design meets its intent (§2.1). In an agent harness, an agent writes to a file in a directory it believes is the target, then verifies by reading the same filename in the same wrong directory. The check passes — *necessarily*, since action and check resolve against the same coordinate. These are not analogies. They are the same defect: **agreement between two things that share a premise is a statement about the premise.**

Second, **the transfer is lossy in a characterisable way**, and the loss has consequences the source framework does not anticipate. This is the part we think is worth writing down. A method that can recompute can treat mutation testing as a *quality check on its own properties*: you mutate the design, you re-derive the truth, you see whether the property noticed. A method that cannot recompute has no such luxury — and so mutation is promoted from quality check to **the only available source of ground truth**. That promotion is not a stylistic difference. It changes what the metric denominator is allowed to be (§4.5, §8), and it changes which verdict is normal (§6.3).

1.2 Why this target

Three reasons, in ascending order of weight.

It is where the failure is costly. Agent reliability is the binding constraint on enterprise deployment, and specifically a kind of unreliability that is hard to see because the agent does not present it as such. A harness whose agents succeed most of the time and report success more often than that is worse than one whose agents succeed as often and report accurately: you cannot build a retry policy, an escalation path, or an audit trail on a signal that does not track the thing it names.

It is measured, and the measurements point at the mechanism. Three 2026 results frame the target (§3.1): confident false completion at 65.4% of failures in one industry evaluation [CITED]; 45–48% of failures in single-control benchmark domains against 3% where an independent simulator can verify state [CITED]; and no LLM-judge configuration above AUROC 0.65 at separating false from real successes [CITED]. The 3% figure is the important one for this paper's purposes: a fifteen-fold difference produced by adding **independence**, not capability.

The instruments are structurally appropriate. The source framework's three tools — falsifiability, same-origin detection, mutation — are all about the *relationship between a claim and its evidence*, not about digital logic. That is why they have a chance of transferring at all, and why the transfer analysis is informative rather than an assertion of analogy.

1.3 What the paper claims, in one line each

Claim	Where	Status
Three of four audit instruments transfer; recomputation does not	§4	argument
Loss of recomputation forces the metric denominator to be the probe suite	§4.5, §8	argument
The taxonomy is generated from one falsifiability condition, not enumerated	§5	argument
False completion is constructible and detectable	§7	demonstrated by demo
The same-origin mechanism in agents is *not* our discovery	§1.4, §3.4	concession
No claim is made about prevalence	§9	scope limit

1.4 Contributions, and what is explicitly not new

We state the boundary first, because overclaiming it would be the exact failure this paper describes.

Not new. Two things. (i) The observation that an agent derives its check from the same reading of the task as its solution — so the check inherits the misreading and passes — has been stated in prose by 2026 industry evaluation work: agents "build against a misreading of the task, then derive a check from that same reading, and the check passes… the check inherits the same mistake and passes anyway" `[CITED]`. (ii) That LLM judges are weak at detecting this is now an empirical result `[CITED]`, not a hypothesis. Anyone reading this paper for the mechanism will find it anticipated, and correctly so.

New, or at least not stated there.

1. A **transfer analysis** of an audit method across a medium change, instrument by instrument, including which instrument *strengthens* on transfer (§4) — a question not raised by work that stays inside one medium.
2. The identification of **loss of recomputability** as the load-bearing difference, and the derivation of two consequences from it: probe-suite denominators (§4.5, §8.2) and `INVALID` as the modal verdict (§6.3).
3. A **taxonomy generated from one falsifiability condition** rather than enumerated (§5), in which the four modes are the four ways a signal can lose its referent.
4. An **audit that opens the harness** (§6) — pipelines, exit-status propagation, state snapshots, path resolution, guardrail predicates — where existing agent measurement treats the harness as a black box.
5. A **four-valued verdict alphabet** (§6.2) in which `INVALID` and `NOT RUN` are distinct from both `PASS` and `FAIL`, and the loss-of-recomputation argument for why `INVALID` dominates.
6. A **worked case of denominator mismatch, committed twice in one figure** (§8.1): a task-completion score reported as a false-completion rate — mode 3 in the number's frame — attributed to an industry report that no search could afterwards locate — mode 3 again, in its source. The second failure is why the case is kept: an unfalsifiable-by-inspection attribution is exactly what this paper is about, and here it was committed by this paper.
7. A demonstration that **the auditor is inside the system audited** (§7.3, §7.4), including three defective checks and one defective mutation test that this author wrote in one session while trying to be careful.

1.5 Vocabulary, fixed here

A **claim** is an assertion the agent produces. An **observation** is a value read from outside the agent's own narration. A **witness** is an observation that would have differed had the work failed. Something is **checkable** if a procedure exists by which a third party could determine it; it is **verified** if that procedure has actually been run by someone. This paper contains checkable claims and no verified ones — we ran no experiments — and we will not use the second word for the first.

A **recomputable** system is one where the audit can independently re-derive the ground truth from public inputs. The distinction between recomputable and non-recomputable systems is the organising axis of this paper, and §4.4 argues it is the only axis that matters.

---

2 The Source System

2.1 What the EquivSVA audit found

The source is our own audit of the public dataset EquivSVA `[CITED]` — 120 behaviour families, four equivalent RTL implementations per family, three controlled mutants per family — in which a formal engine returned `PROVEN` and `PROVEN` was not false. Three findings:

1. **Same-origin generation.** In 80 of 120 behavioural families, the property and the design were both generated from the same `spec.json`, and the generator reproduced them byte for byte. The engine certified agreement between two co-generated artifacts.
2. **Unwitnessed invariants.** 131 invariants received no reachability witness. They were asserted and not exercised; whether they were true was not established by the run.
3. **Unkilled properties.** 79 of 93 formally sound properties were killed by none of 3 controlled mutants. A property that no mutant can kill does not distinguish the design from its mutants, which is the definition of a property with no discriminating power.

The reading that follows is the one this paper carries: **the verdict was sound; the reading was not.** `PROVEN` meant "these two artifacts are mutually consistent", and it was being consumed as "the design meets its intent". The gap between those two readings is the whole subject.

2.2 The three instruments

Extracted from the above, three instruments:

1. **The falsifiability condition.** A property no mutant can kill is not a check, it is a decoration.
2. **Same-origin detection.** Ask not "do these agree" but "do these share a premise". Agreement between artifacts with a common origin is expected and therefore uninformative.
3. **Mutation, not inspection.** You cannot tell by reading a check whether it can fail. You find out by trying to make it fail.

2.3 What made them work, which the paper had not noticed

This is the part the source audit did not make explicit, and the part the transfer exposes.

All three instruments were cheap **because the system was recomputable**. Same-origin detection was not a judgement: the generator was public and deterministic, so "do the property and the design share an origin" was answered by re-running the generator and comparing bytes. Mutation testing was not evidence-gathering: the truth was independently re-derivable, so a mutation experiment was a test of whether a property *tracked* a truth that could be had another way.

In other words, the source framework had an **oracle it did not pay for**. Three instruments, but one substrate: *you can go and find out*. Everything in §4 follows from asking what happens when that substrate is removed.

---

3 The Target System

3.1 The empirical situation

The 2026 work is specific enough to constrain the analysis rather than merely motivate it:

1. In an industry evaluation of Terminal-Bench-aligned tasks, **65.4% of failures ended in confident false completion** [CITED] — the agent ran its own checks, read them as passing, and declared success. The same project's failure-modes page puts the share at **67.2%** of classified failing trials, **2,574** of them [CITED].
2. Whether an agent ran a check was nearly uninformative: **76.4% of passing trials ran one, against 76.8% of failing ones**. Agents asserted success *more often when they failed* (84.3%) than when they passed (80.6%) [CITED].
3. A systematic study across two benchmarks finds false success in **45–48% of failures** in single-control domains, **3%** in a dual-control domain where an independent user simulator can verify state, and **75.8%** among self-assessing coding-agent trajectories. No LLM judge configuration in that study exceeded **AUROC 0.65**; lightweight non-LLM detectors did substantially better [CITED].

We are careful about what these are not. They measure different quantities on different denominators, and a 2026 review observes that consequently **no headline number can be quoted against another** [CITED]. §8.1 takes that observation seriously enough to work a case where violating it produced an inverted figure.

3.2 Why the current responses do not reach it

Three responses dominate, each a reasonable engineering instinct.

Score the output. Build a text-layer metric over the agent's final answer and its claims. This is where the FCR (False Completion Rate) family sits [CITED]: judge whether the reported completion matches the actual state, at the level of what the agent said.

Ask a second model. Use an LLM judge, or ask the agent to audit its own transcript. The 2026 evidence is that this does not work, and that the more carefully it is done the less it works [CITED].

Harden the perimeter. Add guardrails, allow-lists, confirmation gates, output validators.

Each is defensible alone. Together they share the premise this paper rejects:

> **A verdict about an execution can be produced from inside that execution's frame of reference.**

Score the output, and the scorer reads the artifacts the execution produced. Ask a judge, and the judge reads the context the execution assembled — often the very same summarized context. Harden the perimeter, and the guardrail is a function of the same inputs whose trustworthiness is in question. In each case the verdict and the thing judged **share a premise**, so their agreement is a statement about the premise.

The judge result is the empirical form of this argument. If the failure were a capability problem, a better judge would do better. What the study reports instead is that judges fall back on **surface completion proxies** — confident closing language in one benchmark, coarse action-sequence volume in the other — rather than verified state changes [CITED]. A judge reading a narration can only read the narration.

3.3 The four differences that matter

The transfer problem is not "agents are different from circuits". Four specific differences do the work, and each maps to something in §4:

Axis	Source system	Target system	Consequence
Determinism	same input, same output	stochastic	a single run is not a witness
Publicity	inputs and generators public	prompts usually private	cannot re-derive the artifact
Reference	a specification exists	no reference implementation of "what it would have done"	cannot recompute
Observables	tool output	the system's own narration is part of the data	the measurement is inside the system

The third row is the decisive one and §4.4 is devoted to it. The fourth is what makes §6.3's verdict discipline necessary rather than pedantic.

3.4 Which scales share the structure

Because the mechanism is the same defect at three scales, the instrument set has three places to be applied — and recognising it as one defect rather than three is what makes the transfer informative rather than opportunistic.

Scale	Two things that agree	Shared premise	What the agreement proves
Formal verification	property, design	one `spec.json` generated both	the two artifacts are consistent
Agent harness	action, check	one coordinate (cwd, path, version) resolves both	action and check are consistent
Agent self-audit	agent's output, judge's verdict	one context, often one summary of one context	output and the judge's reading are consistent

Credit where it is due. The middle row is not our observation in the agent setting — the 2026 Terminal-Bench analysis states it directly `[CITED]` — and the bottom row is now empirical rather than hypothesised `[CITED]`. What the *transfer* adds is that rows 1–3 are one defect, which is what licenses the instruments to move (§4), and which predicts that hardening (row 2) and adding judges (row 3) fail for the same reason rather than two different ones.

That prediction is falsifiable, and it is the first experiment this programme needs: improve the judge while holding its premise fixed. If accuracy rises materially, the mechanism account is wrong. Our reading of the existing evidence is that it does not rise, but we claim the structure, not the measurement.

---

4 The Transfer

The analysis, instrument by instrument. Each subsection states what the instrument was, what it becomes, and what is lost or gained. §4.6 collects the results.

What is counted as an instrument. Four things are analysed. Three are the instruments the source audit named (§2.2): the falsifiability condition, same-origin detection, and mutation. The

fourth is a property the source audit never called an instrument because it never had to — recomputation (§2.3), the substrate under all three, which becomes visible as a separate thing only when it is removed. §1.3's "three of four" counts these four. The two further rows in §4.6 — the verdict alphabet and the metric denominator — are not instruments; they are consequences of the loss, carried in that table because they belong with it rather than to complete a count.

4.1 Falsifiability — transfers unchanged

In the source. A property no mutant can kill is a decoration. The instrument asks: is there any execution in which this property would report false?

On arrival. The same question, about a different object: is there any execution in which this success signal would report failure? Formalised:

> Let `S` be the success signal the agent consumes for action `A`. `S` is a *check* on `A` only if there exists at least one execution in which `A` fails to achieve its intent and `S` reports failure. If no such execution exists, `S` is a *decoration*.

Why it transfers unchanged. Because it was never about digital logic. It is a statement about the range of a signal along the axis that matters. A property and an exit status are both signals; both can have a degenerate range; the test for degeneracy is the same test.

What is lost. Nothing structural. But note what changes in *cost*: in the source system, "is there an execution that kills it" could be answered by running the mutants you already had. On arrival, the answer requires an execution you must construct (§4.3).

4.2 Same-origin detection — transfers, but degrades to a search

In the source. Compare byte for byte. Re-run the generator, hash both artifacts, observe that the property and the design share an origin. This is a *computation*: it terminates, it is exact, and its result is not open to interpretation.

On arrival. You cannot re-generate the artifact. The prompt is private, the model is stochastic, and there is no canonical "what this harness would have produced". So same-origin detection becomes a **search for a shared coordinate**: does the action and the check resolve through the same cwd, the same inode, the same version, the same environment variable, the same locale?

```
intent : append one line to  <repo>/config.yaml
action : write  "line"  >>  config.yaml     # cwd is /tmp/staging, not <repo>
check  : grep -c "line" config.yaml         # ALSO resolved in /tmp/staging
```

The check passes *because* it resolves the same name against the same drifted coordinate.

The degradation, stated precisely. A byte comparison is exact and total: either the bytes match or they do not. A coordinate search is **neither**. It is exact only relative to the coordinate list you thought to check, and the list is open-ended — cwd, inode, symlink target, file mtime, locale, shell option, installed tool version, clock. You cannot prove the absence of a shared premise; you can only fail to find one. So the verdict on same-origin in the agent setting is `INVALID` far more often than it was in the source setting, and §6.3's alphabet exists partly to keep that honest.

What is gained. A positive finding is still conclusive when you get one: a demonstrated shared coordinate is a demonstration. The asymmetry — hard to prove absent, easy to prove present — is the same asymmetry as in any search problem, and the audit must be designed around it rather than pretending to a totality it does not have.

4.3 Mutation — transfers and strengthens

In the source. Mutate the design; check whether the property notices. A surviving mutant implicates the property.

On arrival. Inject a condition that must fail; check whether the signal notices. A probe that does not surface implicates the harness.

```
# [SEMANTICS] the probe is constructed, not discovered
find /nonexistent_probe -name '*.log' 2>/dev/null | wc -l    # prints 0, exit 0
```

Why it strengthens. This is the surprising result of the transfer, and it is worth stating clearly.

In the source system, mutation required a **correct reference design** to mutate. The mutants had to be *plausible variants of something correct*, or the experiment said nothing about the property's discriminating power — an arbitrary broken mutant is killed by everything.

On arrival, no correct reference is needed. A probe only has to be a condition that **must** fail, and such conditions can be synthesised from nothing: a path that does not exist, a write to a read-only location, a guardrail-violating input. The absence of a reference implementation — which cripples §4.2 — is exactly what makes probe construction cheap here.

So the instrument that degrades and the instrument that strengthens are degraded and strengthened by the same missing piece. That is not a coincidence; it is the shape of the loss.

What strengthening does not buy. There is an obvious objection here, and it should be answered rather than left to the reader. In the source system an arbitrary broken mutant is killed by everything, which is precisely why the mutants had to be plausible variants; a path that does not exist, a write to a read-only location, a guardrail-violating input are all conditions that *obviously* fail. If obvious mutants were uninformative there, why are obvious probes informative here?

The answer is that the two experiments ask different questions. A mutation experiment asks whether a property discriminates among *plausible* behaviours, which is why the mutant must be plausible. A probe asks whether the harness can carry a failure from the point of failure to the point of reporting, which is why the failure must be undeniable. The second question only appears trivial because its answer is assumed; in the systems §5.2 describes, the signal is lost in transport, so a condition that obviously fails is exactly the condition that distinguishes a harness which reports from one which cannot. The probe tests the *pipeline*, not the agent's discrimination.

That is a necessary condition for detecting subtle failure, not a sufficient one. A suite of undeniable probes cannot show that an agent recognises a subtle target drift (§5.4); it can only show that, if the agent did recognise one, the report would survive the trip out. The stronger test needs a level-2 witness, and the witness is limited by the same loss of recomputation (§4.4). So the instrument is weak, and its weakness is not fixable here — but "weak" and "unnecessary" are different verdicts, and in a non-recomputable system the necessary condition is usually the only one available.

4.4 Recomputation — lost, and this is load-bearing

In the source. Re-run the generator. Re-derive the truth. Compare.

On arrival. Impossible. There is no reference implementation of "what this harness would have done", because the harness contains a stochastic model and often a private prompt. You cannot go and find out.

We want to argue that this single loss is what characterises the whole transfer, and that its consequences are not obvious in advance.

Consequence 1: mutation is promoted from quality check to evidence source. In the source framework, mutation answered a *second-order* question — "does my property track a truth I can obtain otherwise?" The truth was available independently. On arrival there is no otherwise, so injection becomes the **only** source of ground truth. The instrument is unchanged; its epistemic role is not. A mutation suite in the source is a quality gate. A probe suite on arrival is the **measurement instrument itself**.

Consequence 2: this forces the denominator. If the probe suite is the only source of ground truth, then no metric can have a workload denominator. An "execution integrity rate over real traffic" would need an oracle for "this invocation actually failed", which is precisely what the loss removed. §4.5 derives the metric consequence; §8.2 states it.

Consequence 3: `INVALID` stops being an edge case. In a recomputable system, "could not evaluate" is rare — you can almost always re-run. Where you cannot, it is an anomaly worth investigating. On arrival, "could not evaluate" is the **default** state of most audit questions, because most questions need a witness you cannot obtain. A verdict alphabet without `INVALID` will therefore not report an excess of `INVALID`; it will report an excess of `PASS`, because everything unevaluable has to go somewhere. §6.3 develops this.

What is *not* lost. Recomputation's absence does not license guessing. It changes what counts as evidence, from "an independently re-derived fact" to "an observation read through a channel the action did not write to", which is the level-2 witness of §6.1. This is a weaker standard, and the paper's central methodological claim is that it can still be met — not that it is equivalent.

4.5 The consequence for measurement

Derived, not asserted.

	Recomputable (source)	Non-recomputable (target)
Ground truth from	re-derivation	injection
Denominator can be	the corpus	only the probe suite
Metric means	a property of the artifact	a property of the suite
Unevaluable cases	rare, anomalous	common, expected
Normal verdict	`PASS` or `FAIL`	frequently `INVALID`

The third row is the one that must be said out loud, because it is where this kind of metric gets misused. If the denominator is the probe suite, the metric is a statement **about the suite**, not about the system's behaviour in deployment. A harness can score perfectly on twenty probes and fail on the twenty-first. That is not a flaw in the metric; it is what the metric is.

The fourth and fifth rows are the same fact seen from the verdict side, and they explain why §6.3's inequalities are not pedantry: in a domain where most questions are unevaluable, the difference between "unevaluable" and "fine" is most of the information there is.

4.6 The transfer, collected

Instrument	Source form	Target form	Status
Falsifiability condition	property no mutant kills is a decoration	signal that cannot report failure is a decoration	**unchanged**
Same-origin detection	re-derive from public bytes and compare	search for a shared coordinate	**degraded**: exact→open-ended
Mutation	mutate a correct design	inject a must-fail condition	**strengthened**: no reference needed
Recomputation	re-run the generator	impossible	**lost**
Verdict alphabet	0 / 5 / 2 / 1	`PASS` / `FAIL` / `INVALID` / `NOT RUN`	**unchanged**
Metric denominator	the design corpus	the published probe suite	**forced by the loss**

Three move, one dies, and the one that dies reorganises the rest. That is the paper's structural claim. Everything after this point is the transferred method being applied.

---

5 The Taxonomy

5.1 The generative principle

Four failure modes are a taxonomy only if they are generated rather than enumerated. The generator is the transferred falsifiability condition of §4.1.

Ask: in what distinct ways can a genuine signal stop satisfying "there exists an execution in which it would report failure"? There are four, and they are not arbitrary — they exhaust the ways a signal can lose its referent.

Mode	Dimension	The signal is…	Why the condition fails
1. Masked failure	transport	produced correctly, then destroyed	it never reaches the reader
2. Stale state	time	real, but about a prior execution	it is current for the wrong moment
3. Target drift	identity	real and current, about another object	it is about the wrong referent
4. Tautological guardrail	predicate	produced, read, and passing	it cannot take the value "failure"

Transport, time, identity, predicate. Any signal needs a carrier, a moment, a referent, and a range; remove any one and the signal is a decoration.

5.2 Mode 1 — Masked failures (transport)

The canonical carrier is the Unix pipe, whose exit status is that of its **last** command:

```
# [SEMANTICS] reproducible in one line
find /nonexistent | wc -l      # prints 0, exit status 0 — find failed, wc succeeded
```

`find` fails with status 1; `wc` succeeds with status 0; the pipeline's status is 0, and a harness that reads it has observed success. This is not an edge case; it is the default behaviour of the most common composition operator in the tool surface.

`pipefail` and `PIPESTATUS` are the remedies, and the remedy has a trap that is itself a **second-order** masked failure — `PIPESTATUS` is overwritten by the very next command, which need not look suspicious:

```
# [SEMANTICS] verified for this paper
false 2>/dev/null | cat >/dev/null; printf '%s\n' "${PIPESTATUS[0]}"          # 1
false 2>/dev/null | cat >/dev/null; m=$?; printf '%s\n' "${PIPESTATUS[0]}"    # 0
```

An assignment `m=$?` — the most natural way to save a status — destroys the witness it was saving. We found this by writing the demo in §7.4 and watching it fail.

5.3 Mode 2 — Stale state (time)

The signal is genuine and correctly transported; it belongs to an earlier execution. The reasoning is sound given its premise, and the premise is a photograph of a world that has moved.

Carriers: **resumed runs** (a checkpoint read at step k though the environment changed at $j < k$ — the cache is not wrong, it is old); **unreset accumulators**; **compacted memory** (an earlier successful step becomes a summary sentence, which is a claim about a past observation and loses the observation, including the ability to notice it was of a different world); **retained artifacts** (a results

file from a previous run, read because the current run failed to overwrite it — the reader cannot distinguish "this run produced this" from "this file exists").

The distinguishing feature: **everything checks out.** No transport bug; the value was delivered faithfully. The audit must compare the signal's *origin time* against the action it certifies — a check no amount of reading the value can perform. This is a case where §4.4's loss is felt directly: without recomputation, the audit cannot establish what the environment was at time *t*, only what it is now.

5.4 Mode 3 — Target drift (identity)

The action succeeded, the signal is fresh and faithfully transported, and it is about the wrong object. The write landed; it landed elsewhere.

- **Working-directory drift.** A relative path resolves against a cwd that is not the intended one. We met the degenerate case in the EDA work: a shell session's cwd had been removed, so every subsequent command failed to spawn at all `[OBSERVED]` — extreme, but it makes the point that cwd is part of the identity of every relative operation while being almost always invisible.
- **Scope inflation.** The object is larger than the intent: a whole-file write where append was meant; a recursive delete where a single-directory delete was meant. `[OBSERVED]` One file-writing tool call replaced an entire manuscript when an append was intended; the tool returned success, which was true, about the wrong thing.

What makes this mode survive review is that **the verification drifts with the action** — the §4.2 degradation in its practical form. The remedy is structural: the check must resolve its object by something **independent of the action's own resolution** — an absolute path, an inode, or a digest captured before the action.

Two further observations:

The whole file is a live object. `[OBSERVED]` A human-written report twice carried a stale line count after the numbers were mechanically reconciled — prose said "637" while the file had grown to 652. Neither file nor sentence was wrong when written; the sentence became wrong by staying still. Modes 2 and 3 at once.

Cleanup is not neutral. A "clean the scratch directory" step resolving against a drifted cwd deletes something else and reports success — accurately, about the wrong tree.

5.5 Mode 4 — Tautological guardrails (predicate)

The signal is produced, transported, fresh, and about the right object. It also cannot fail.

Carriers: **hardcoded truth** (`if True:`, a comparison commented out during debugging, a config flag that defaults to skipping the check); **vacuous predicates** (a check over an empty set — `all([])` is `True`, a loop over zero rows reports no violations, and there was nothing to object to because it was looking at nothing); **never-mutation-tested validators** (this is the source framework's mutation finding transplanted exactly); **self-certifying checks** (the check imports the same constant as the code under test, so it cannot disagree).

The inference "the guardrail returned clean, therefore the action was safe" holds only if the guardrail could have returned dirty. That premise is almost never checked, and by §4.3 it is the cheapest of the four to check.

5.6 Composition, and the guardrail paradox

Composition	Result
Masked failure inside a tautological guardrail	undetectable by construction — the rejection path would have masked the error anyway
Stale state plus target drift	confident, specific, wrong: a fresh-looking value about the wrong object
Target drift inside a retry loop	each attempt "succeeds", so retries never trigger
Any mode plus memory compaction	the failure becomes a summary sentence asserting success

The first row generalises into a claim running against the consensus on hardening:

> Adding guardrails is not monotonically safety-improving. A guardrail that cannot fail, or that reports through a transport that can lose its failure, **converts a visible failure into an invisible one** — and consumes the attention that would otherwise have caught the problem.

The mechanism is mode 1 plus mode 4: a guardrail is itself an execution with an exit status and an output. If its rejection path is never exercised, it is inert; if it is exercised but its message is piped through a summariser, the rejection is destroyed. Either way the observable is the same as success, and the system now has one more component asserting soundness.

This is the reported result explained. Self-checks ran at nearly identical rates in passing and failing trials (76.4% vs 76.8%) and agents asserted success more when failing `[CITED]` — which is what a system composed of near-tautological checks should produce. `vacuous_pass` in the source audit — a check that passed because it never engaged — was worse than no check, because it consumed the trust that would otherwise have been spent on the real gap.

---

6 The Audit Procedure

Three levels, ascending in cost and strength of evidence. The ordering is not arbitrary: it is the ordering of §4.4, from what survives the loss to what it forces.

6.1 The three levels

Level 1 — evidence origin. *Did the fact the agent asserted come from the environment, or from the agent?* Instrument: a **provenance trace** joining each claim in the transcript to the observation it rests on and the tool call that produced it. A claim with no tool call behind it is a claim about the agent's own state. Cheap and mechanical — the trace is a join, not a judgement. Catches mode 4 and the narration half of mode 2. Cannot catch a fact read faithfully from a stale or wrong object.

Level 2 — execution witness. *Is there an effect outside the transcript?* A witness is an observation that would have differed had the work failed: file digest before and after, a row count from the database rather than from the agent's summary of it, an exit status captured out-of-band. Design rule: **the witness must be read through a channel whose value is not determined by the action's own narration or the action's own resolution.** This is the operational content of "independent", and it is what fails in modes 1 and 3. It is also the level most exposed by §4.4: without recomputation, the witness is the *only* thing that can stand in for ground truth, and it must be captured live because it cannot be re-derived later.

The rule is stated in terms of determination rather than in terms of writing, because an action can leave a channel unread and still determine its value: an exported environment variable, a changed working directory, a touched mtime, a file written by a subprocess the action launched. "The action

did not write to it" is therefore too narrow a test, and a witness that satisfies it can still be the action's own echo. The corresponding test in §4.2 is the same one: does the witness resolve a coordinate that the action also resolves?

The empirical warrant: the 3%-vs-45–48% gap in §3.1 is produced by exactly this — a dual-control domain in which an independent party can read the state. Level 2 is an attempt to buy that independence cheaply.

Level 3 — mutation probing. *Can the pipeline report failure at all?* Inject a condition that must produce a failure and observe whether it surfaces: a command that exits non-zero; a guardrail-violating input; a write to a read-only location; a stale artifact planted in the cache. A probe that does not surface is a **positive finding** — a constructive demonstration that the signal is a decoration.

By §4.3 this level is the strongest available, because it is the only one that does not depend on obtaining ground truth from elsewhere. And by §4.5 its cost is that the probe suite becomes the denominator.

6.2 The verdict alphabet

An audit must return four things, and must not collapse them into two.

Verdict	Meaning
PASS	the audit ran; the levels held under the probes applied
FAIL	the audit ran; at least one level did not hold
INVALID	the audit ran, but a level could not be evaluated — no witness obtainable, or no probe constructible
NOT RUN	the audit did not run — no harness access, no transcript, no environment

A verdict a naive alphabet collapses. The failure mode this alphabet exists to prevent is returning `PASS` for an audit that could not be completed. One case needs stating explicitly, because it looks like a finding and is not.

Same-origin, not found. The coordinate list is an open set, so failing to find a shared premise is not `PASS`: absence of the finding is not a finding of absence. It is also not `FAIL`, because not found does not mean absent. The default is `INVALID` — unless the audit can certify that the coordinate list was complete for the question class at issue. That certification is rare and is itself hard to audit, which is why §4.2 expects `INVALID` to dominate. This is the level-1 counterpart of the probe-suite adequacy problem in §9: in both cases the audit's answer is relative to a set whose completeness is assumed rather than shown.

6.3 Why three inequalities, and why the fourth verdict dominates here

1. **INVALID $\neq$ PASS.** "We could not construct a witness" is not "there was no problem". A vacuous pass is the most expensive error, because it consumes trust.
2. **NOT RUN $\neq$ FAIL.** "We did not test it" is not "it is broken". Collapsing this direction produces false alarms and, over time, an audit nobody believes.
3. **Neither is evidence in the other direction.** `INVALID` neither supports nor refutes the claim under audit; it records that the question was not answered.

The alphabet is lifted from the source audit, where exit codes were separated as 0 = pass, 5 = not pass, 2 = nothing judged, 1 = driver crashed, with the explicit warning that reading 2 or 1 as "does not pass" turns "not tested" into "tested and false", or a crash into a verdict.

What the transfer changes. In the source, `INVALID` was an anomaly. Here, by §4.4 consequence 3, it is the **normal** state of an audited question: most questions need a witness that cannot be

obtained, or a probe that cannot be constructed. This has a practical consequence that is easy to miss: an audit implementation without an `INVALID` value will not report an unusual number of `INVALID`s. It will report an unusual number of `PASS`es, because every unevaluable question must be filed somewhere, and the path of least resistance is "no problem found". In a domain where most questions are unevaluable, that single design choice converts the alphabet from an instrument into a laundering mechanism.

From the source audit, the parallel is exact: agent audits are consumed by downstream automation, and automation that cannot distinguish `INVALID` from `PASS` will silently convert gaps into green.

Open problem, stated rather than hidden. There is no cheap way to certify that a *given* transcript was produced by the run being audited, as opposed to a previous run — the run-identity gap registered in the source audit. Absent that, level-2 witnesses must be captured live and timestamped, and a transcript without live witnesses is at best level-1 evidence. This is mode 2 at the level of the audit itself, and it is a case where §4.4's loss reaches all the way up into the methodology.

---

7 Cases

Illustrative, not experimental. No measurement is reported; outputs shown are what the semantics dictate, verified for this paper where tagged.

What these cases are instances of. They are not independent evidence, and they are not meant to be read as a survey of how agents fail. Each one instantiates a failure mode that §3.1 already measured: the 65.4% false-completion figure, the 76.4%-vs-76.8% check-running rates, and the 45–48%-vs-3% control-domain gap are the prevalence claims, and §5 is the mechanism account. The cases exist to make that mechanism operable — to show what the failure looks like in a pipeline, in a report, in a path resolver, in a mutation test — so that a reader can look for it. Their role is to make the taxonomy executable; it is not to establish that the taxonomy is common. Prevalence is a separate claim, and §9 declines to make it.

7.1 Case A — a pipeline that eats the failure

```
# [SEMANTICS] The harness sees 0 in both branches.
run_check() { find "$1" -name '*.log' 2>/dev/null | wc -l; }

run_check /data/logs        # exists: prints the match count, exit 0
run_check /data/nope        # missing: prints 0, exit 0   <-- failure masked
```

The agent receives `0` with no error and no non-zero status. Its available inferences are "no log files", "the glob did not match", and "the command never ran" — and it has no information distinguishing the third. In practice it selects the first, the reading that supports continuing.

Two one-line repairs, and the difference is instructive:

```
set -o pipefail                    # pipeline status becomes the rightmost non-zero status
find /data/nope -name '*.log' >/tmp/o 2>/tmp/e; st=$?; wc -l </tmp/o   # capture status first
```

The second is preferable inside a harness, because it does not rely on a shell option a downstream wrapper may reset. Either way the repair is at the *transport* layer: nothing about the agent's reasoning changed, and nothing needed to.

7.2 Case B — verification that drifts with the action

```
cwd = "/tmp/staging"                       # drifted; the agent believes it is at <repo>
append("config.yaml", "feature_x=on")      # succeeds, writes /tmp/staging/config.yaml
ok = "feature_x=on" in read("config.yaml") # also reads /tmp/staging/config.yaml
report("Task Completed" if ok else "Failed")   # reports Completed
```

Both statements resolve `config.yaml` against the same drifted `cwd`, so the check is *guaranteed* to agree with the action. The agent produced a true statement about the wrong file — **by verification**, not by assertion. The repair is to make the witness resolve independently:

```
target = Path("/repo/config.yaml").resolve()        # absolute, independent of cwd
before = sha256(target) if target.exists() else None
append(str(target), "feature_x=on")
after  = sha256(target)
report("Completed" if after != before else "Failed")
```

A check sharing the action's coordinate cannot detect the action's coordinate error — in either domain.

7.3 Case C — the auditor was the failure

From preparing the sibling EDA audit, same session, by this author. Three checks, all wrong, all **reporting success**:

Check as written	What it actually did	Mode
`find . -name '*[^ -~]*'` to detect non-ASCII paths	matched **pure-ASCII names too**, so it could never report a clean package. The pattern is logically correct; the failure was an unexamined premise — under `zh_CN.UTF-8` the range is resolved by locale collation, and `xyz.txt` matches `[^ -~]`	4, always-fail direction
`unzip -l ⋯ | grep -c supplementary` to test "does the archive contain X"	counted the listing's **header line** as an entry, returning 1 where the answer was 0; `grep -c` counts matching lines, not occurrences	4
a loop printing how many manifest entries existed	printed a count and **compared nothing**; would report success on any input	4, pure

The first row is worth more than the other two combined, because it does not have the shape we expected. The pattern was right — `[^⋯]` negation works; we checked separately that `*[^a]*` matches `---`. What broke was the **locale**, a premise nobody thought about:

```
# [SEMANTICS] verified for this paper; fixtures: xyz.txt abc.txt 中文.txt
LC_ALL=C           find . -maxdepth 1 -name '*[^ -~]*'   # only 中文.txt
LC_ALL=zh_CN.UTF-8 find . -maxdepth 1 -name '*[^ -~]*'   # 中文.txt + xyz.txt + abc.txt
```

Three consequences: (i) the audit's correctness depended on an ambient setting it did not record, so a reader reproducing it on `C.UTF-8` would see it behave correctly and conclude it is fine; (ii) the remedy has the same shape as §5.4's — resolve by an identity independent of the ambient frame, an absolute path there and an explicit `LC_ALL=C` here; (iii) it fails in the direction that destroys trust in the tool, since always-fail is as uninformative as always-pass and teaches the reader to ignore the check.

The third row is mode 4 in its purest form: a predicate that was never written. It ran, produced a number, and the number was read as agreement.

7.4 Case D — the demo, and the mutation that did not mutate

`agent_audit_demo.py` (shipped alongside this paper) implements Case A and the §6.1 level-3 probe idea in under 340 lines using only the Python standard library. A harness pipes a command's output through a formatter; an agent reads the resulting status; the auditor captures **both** statuses from a single execution:

```
cmd 2>/dev/null | cat
printf '%s|%s' "${PIPESTATUS[0]}" "${PIPESTATUS[-1]}"    # true | masked
```

Two details here are load-bearing, and the demo's comment says so. The masked status is the **last** stage, not stage 1: when `cmd` itself contains a pipe, `${PIPESTATUS[1]}` is that inner command, not the formatter, and reading it turns a masked failure into a `PASS` — a false negative found by attacking the instrument rather than by reading it (`false | false` reports masked = 1 and swallows the failure). And the `printf` must carry a leading newline, or a command that emits no trailing newline fuses its output with the status payload and the parse fails.

A single execution matters. Re-running the command to recover the true status would be a second execution, i.e. mode 2 at the level of the audit itself.

Four mutants, with expected verdicts fixed before running:

Mutant	Expected	Why
masked failure	`INVALID`	agent saw 0, command returned 1
same command, transport fixed	`PASS`	both statuses 1, and the agent says so
command genuinely succeeds	`PASS`	both statuses 0
masked, agent also wrong about the outcome	`INVALID`	statuses disagree

Then the part that makes it an audit rather than a demonstration: **the auditor itself is mutated.** We wrote four, and one of them did not mutate anything.

Mutation of the auditor	Effect	Caught?
replace the primary guard clause with `if False:`	**no change** — a second guard clause still caught the case	no, and correctly: the auditor was unchanged
return `PASS` unconditionally	predicate removed (mode 4)	yes — 2 of 4 mutants mismatched
return `INVALID` unconditionally	always-fail, equally useless	yes — 2 of 4 mismatched
capture status with `m=$?` instead of `PIPESTATUS`	witness destroyed (§5.2)	yes — 2 of 4 mismatched

The first row is the one worth keeping. **Our mutation test was itself a decoration.** Disabling one guard clause did not disable the audit, so "4/4 mutants behaved as required" was correct and uninformative: it reported that an unmutated auditor behaves like an unmutated auditor.

This is mode 4 occurring inside the tool built to detect mode 4, and it has a specific consequence for practice: **a mutation test must assert that the mutant changed the observable behaviour, not merely that the suite ran.** A surviving mutant is informative only if the mutant was alive to begin with.

The general rule all of §7.3–7.4 produces:

> A check is admissible only after it has produced a **failing** verdict on a known-bad fixture.

This is the §4.3 instrument turned on the audit itself, and by §4.4 it is non-optional: with no recomputation available, the audit cannot otherwise distinguish its own success from its own silence.

---

8 Measurement

8.1 The denominator mismatch, worked in full

This subsection exists because the paper argues that denominators are where this literature breaks, and it would be a poor argument if made only in the abstract. Here is a case where it broke: in an earlier draft of this paper, in a way that inverted a number's meaning.

Where the case sits. An earlier version of this subsection described the break as having occurred in the circulated secondary literature. That description is withdrawn, because it cannot be supported: a search for a secondary source reporting 77.4 as a false-completion rate located none (Appendix C records the attribution as `MISATTRIBUTED`). What can be documented is the version of the error that certainly existed — in this paper's own earlier draft — together with the attribution it carried to an industry report that no search could find. The example is not weaker for being ours. It is stronger, because the same failure appears twice in it: a number changed frames, and a source was named that could not be checked.

The number. A figure of **77.4%** was carried into an earlier draft of this paper as a false-completion rate, attributed to an industry "Trust Layer" report.

What it actually is. 77.4 is a **Task Goal Completion** score: the score of one method (ASSAY, with GPT-3.5) on a public AppWorld leaderboard `[CITED]`. It measures how much of a task's goal was completed. Higher is better.

The two frames.

Property	Task Goal Completion	False Completion Rate
Measures	how much of the task was done	how often a success claim was false
Direction	higher is better	**higher is worse**
Denominator	tasks attempted	failures, or claimed successes
A value of 77.4 means	the method did most of the task	four out of five completions were lies

Neither number is wrong. They are near-inverses of one another, they both live in the agent-evaluation literature, both are expressed as percentages, and both are described as being "about whether agents complete tasks". A number moved from the left column to the right and kept its plausibility while losing its sign.

Why it survived review. Three properties, and each is worth naming because they are general:

1. **Same vocabulary.** "Completion" appears in both names. The migrating number never had to change words, only meaning.
2. **Plausible magnitude.** ~77% is an alarming-sounding false-completion rate and a respectable task-completion score. The figure did not look wrong in either frame, so no frame triggered suspicion.
3. **Attribution to a plausible source.** "Industry report, 2026" is an unfalsifiable-by-inspection attribution. It made the number checkable in principle and checked in practice by nobody.

What it illustrates. This is mode 3 — target drift — in a paper about mode 3. The number was fresh, faithfully transported, correctly quoted as characters, and about the wrong object. The verification (the citation) drifted with the claim (the figure), which is precisely the §5.4 structure: the check resolved the same coordinate as the action. That the drift happened here, rather than only in the work this paper examines, is the reason the case is included: it is not an illustration of someone else's error but a record of this one.

The discipline it implies. Not "be more careful". Carefulness did not fail here; *searching* is what found it. A denominator is a claim, and unlike a numeric value it has no typo to catch — it is either the right frame or a different question. Consequently:

> A percentage without its denominator and its direction is not a weak fact. It is a fact about a different subject.

This is why the metrics defined in §8.3 state their direction in the table rather than in prose, and why the State Drift Rate is marked inverted. A composite dashboard that averages the Execution Integrity Rate and the State Drift Rate will report a drift-prone harness as healthy — mode 3, generated by the metric layer itself.

The same defect, committed here. An earlier draft of §3.1 read "67.2% over 2,574 failing trials". Both numbers are on the source page and both are correct — 67.2% is the share of classified failing trials, and 2,574 is how many fell in that class — but written with "over", a numerator was read as a denominator. Nothing was fabricated and the `[CITED]` tag was accurate; a number changed role. In the final verification pass a proposed correction to this subsection asserted that 77.4 belonged to a newer model version; re-fetching the leaderboard showed `ASSAY` with `GPT-3.5`, so the correction was the ungrounded claim, not the figure. Both incidents are mode 3, and both were caught by fetching the source rather than by rereading the draft.

8.2 What these metrics are not: a text-layer FCR

`FCR [CITED]` measures a property of the **agent's output**: over a benchmark, the rate at which reported completions are false. Our metrics measure a property of the **harness**: whether, under injected conditions that must fail, the failure surfaces.

	`FCR`	Metrics here
Unit of measurement	a task outcome	a probe injection
Denominator	tasks in a benchmark	probes in a published suite
What a good score means	the agent's claims were true more often	the harness *can* report failure
What it cannot tell you	why the claim was wrong	how often the agent is wrong in deployment

An FCR of zero is compatible with a harness that swallows every exit code, provided the tasks happened to succeed. Conversely a harness with perfect status propagation can have a poor FCR if the model reasons badly. **They measure different layers, and neither dominates.**

Note the transfer consequence (§4.5): the different denominators here are not a stylistic choice between two metrics. `FCR` can use a task denominator because it has ground truth from a benchmark's own oracle. Our metrics cannot, because §4.4 removed the oracle — so their denominator must be the probe suite, and the cost is that they are statements about the suite.

That `FCR` currently has no reported value is worth recording: the metric is defined and implemented in an enterprise benchmark `[CITED]`, but the same repository's implementation-status table lists the agent adapter and the first end-to-end experiment as **in progress**, and no `FCR` value is published. (A name collision exists with at least two unrelated ACL benchmarks sharing the name, whose reported metrics are pass rates and aggregate scores rather than a false-completion rate `[CITED]`; we mean the `FCR`-defining one.) The definition is the contribution; the measurement is outstanding.

8.3 The three metrics

Let n be the number of probes injected for a class, and k the number that behaved as required.

Metric	Definition	Direction	Probe class
Execution Integrity Rate (EIR)	k / n, where a failing command surfaces as failing	higher is better	commands that must exit non-zero
State Drift Rate (SDR)	k / n, where a stale artifact is *not* consumed	**higher is worse**	planted stale artifacts
Mutation Resistance (MR)	k / n, where a guardrail-violating input is rejected	higher is better	guardrail violations

8.4 Three commitments that matter more than the formulas

1. **The denominator is the probe suite, not the workload.** Derived in §4.5, not chosen. A passive EIR over real traffic would need an oracle for "this invocation actually failed", which is the thing the transfer took away. The restriction is the price of definability, and it makes the metric a statement about the suite — which §9 records as a limitation rather than conceals.
2. **Absent probes yield `undefined`, not `1.0`.** If no guardrail-violating input was ever injected, MR is not 100%; it is unmeasured. Reporting 100% is the metric-layer form of `INVALID` $\longrightarrow$ `PASS`, and by §6.3 it is the most likely way this index will be misused.
3. **The directions differ, and SDR is inverted.** Per §8.1, a percentage without its direction is a fact about a different subject.

8.5 Boundary

This is an audit instrument, not a safety shield. It does not assert an agent is trustworthy; it bounds what was tested. A high MR over twenty probes says the guardrail rejected twenty violations, not that it rejects the twenty-first. Published as a number without its suite, it is exactly the decoration §5.5 describes.

---

9 Threats to Validity

Construct validity of the transfer. We characterise the source/target difference along four axes and claim the third (recomputation) is decisive. A reviewer might argue another axis is: stochasticity, for instance, arguably undermines the reproducibility of the probes themselves. We think recomputation is the right one because it alone changes what a metric is *allowed to mean*, whereas stochasticity changes only variance — but this is an argument, not a result, and the transfer analysis is falsifiable in the way taxonomies are not: if a fourth instrument is found that transfers with consequences we did not derive, the analysis is incomplete.

Construct validity of the taxonomy. The four modes are proposed as exhaustive along the dimension of losing a referent, not as an empirical partition. They are not mutually exclusive, and §5.6 is a sketch, not a calculus. We consider the framework falsified if a false completion is exhibited that cannot be described as a signal losing its carrier, its moment, its referent, or its range.

Mechanism proof, not prevalence measurement. The central scope limit. We report no rate of false completion and conduct no measurement. The `[CITED]` figures in §3.1 are other people's measurements on other people's denominators; we cite them as motivation, not as evidence for this framework. Existence and detectability is a different claim from frequency, and only the first is made here.

Limited novelty, stated plainly. §3.4's middle row has been said in prose by 2026 industry work `[CITED]`, and the bottom row is now an empirical result `[CITED]`. A reviewer who takes same-origin

complicity in agents as this paper's thesis will find it anticipated, and correctly so. The defensible claim is the transfer analysis and its consequences (§1.4).

The probe-suite problem. `EIR/SDR/MR` are only as informative as their probe suite, and no method is offered for certifying adequacy — only for publishing. This is the sharpest limitation, and it is not reparable within the framework as presented: by §4.4 there is no independent ground truth against which to validate the suite. A suite of easy probes inflates every number, and the framework cannot detect it. We prefer to state this as an open hole than to imply a rigour the loss does not permit.

The demo's probe has the same shape of hole. `agent_audit_demo.py` reads the verdict from an instrumented pipeline, and that instrument is itself a probe of one execution. Attacking it with commands that do not appear in this paper found three defects — a masked failure reported as `PASS` when the audited command contained its own pipe (`false | false`), a parse failure when the command emitted no trailing newline, and an uncaught decode error on non-UTF-8 output in which the audit **died instead of returning a verdict**. All three are fixed and recorded here, because finding them required running the instrument rather than reading it.

That set is not a coverage argument. It exercises redirection, command chains, subshells, multi-stage pipes, `set -e`, traps, timeout wrappers, background jobs and binary or very large output, and in sampling each of those returned a verdict — but *returning a verdict* is not the same as *being correct on that case*, and we have no mutation suite over the shell-command space to establish the latter. The demo is a proof of concept for the audit layer, not a production sandbox, and a reader who needs one should not read this one as it.

External validity. The analysis targets code- and tool-using agents, because those have an execution layer to audit. For a pure-conversation agent, mode 3 is largely inapplicable (no object to drift from) and mode 1 is confined to the retrieval layer. Modes 2 and 4 still apply, and we suspect mode 4 is *more* common there, since a conversational guardrail has no side effect to expose it — but "we suspect" is the correct strength, and it is not evidence.

The citation debt, and what checking it changed. An earlier draft carried five figures labelled unverified. Checking them (Appendix B.2) found one misquotation, one misattribution of an inverted quantity, and one citation with no locatable source; two held, one of them with a caveat. The misquotation also carried a numerator where a denominator belonged (§8.1). The repair in every case that survived was to fix or drop the figure, not to weaken the argument — because the argument does not depend on magnitudes. One *proposed* repair did not survive: it would have changed a correct attribution on the strength of a stale memory of a live leaderboard, and re-fetching the page is what rejected it (§8.1). Proposing a correction is itself a claim, subject to the same check. The residual debt is *depth* of verification: several sources were read at vendor-page or secondary-review level rather than as full papers (Appendix B.3).

The auditor is inside the system being audited. We take this most seriously, because we have direct evidence. The three defective checks in §7.3 and the defective mutation test in §7.4 were written by the present author, in one session, while explicitly trying to be careful. Every claim in this paper is checkable, none is verified, and the number of errors we found in our own instruments should be read as a floor on the number remaining.

Detection is not prevention. Every method here identifies false completion after or during the fact. None makes an agent correct. The framing — auditing execution-layer claims — is narrower than "agent reliability", and the narrower claim is the one we can support.

---

10 Conclusion

The useful thing about auditing a non-recomputable system is that it forces you to say what you actually have.

In the recomputable source, we had more than we had noticed: an oracle we did not pay for. Because the generator was public and deterministic, "do these two artifacts share an origin" was a computation, and a mutation experiment was a check on a property whose underlying truth was

available by other means. The instruments looked like three, and rested on one substrate.

Take the substrate away and the instruments do not all fail equally. The falsifiability condition survives untouched, because it was never about specifications in the first place — it is a statement about the range of a signal. Same-origin detection survives as a search, which is weaker in a way that matters: it can demonstrate a shared premise and can never establish their absence. Mutation survives and, against expectation, gets *stronger*, because a condition that must fail can be synthesized from nothing — no correct reference required. What does not survive is recomputation, and it is the only one whose loss reorganises the rest.

Two consequences follow, and they are this paper's practical yield. First, injection stops being a quality gate and becomes the only source of ground truth; therefore no metric can have a workload denominator, and the denominator must be a published probe suite — which makes the metric a statement about the suite, and we say so rather than hide it in a footnote. Second, `INVALID` stops being an edge case and becomes the normal state of an audited question, which is why an implementation without an `INVALID` value will not over-report `INVALID` — it will over-report `PASS`, because every unevaluable question has to be filed somewhere and "no problem found" is the path of least resistance.

We therefore want to state what execution-layer auditing is, in a sentence that resists being read as a promise. It does not make a model more capable, more careful, or more honest, and it does not raise the rate at which agents succeed. **It puts a criterion that can fail where there was previously a report that could only succeed.** That is a smaller claim than "agent reliability" and a larger one than a benchmark score: it does not tell you the agent is right, it tells you whether the evidence for "right" existed at all — and it does so on a channel the agent did not write to. In a setting where the agent's own account is the most available and least trustworthy evidence, that distinction is most of what an auditor can honestly offer.

The framework's own record is the reason to state this modestly. Three of our checks reported success while being unable to fail. One of our mutation tests reported a green suite without mutating anything. One of our cited figures turned out to be its own inverse. Each was found by the same operation that the framework prescribes — an injection into something that would otherwise have agreed with itself — and each was found in our own work, in one session, while we were explicitly trying to be careful. We report that not as confession but as calibration: the count should be read as a floor.

Appendix A — Future work

1. **Test the §3.4 prediction.** The paper's central falsifiable claim: compare a judge reading the agent's narration against a judge reading out-of-band witnesses, on the same tasks. If the second does not outperform the first, the mechanism account is wrong.
2. **Test the §4.4 claim directly.** If a semi-recomputable target existed (a harness with a public, deterministic prompt and a fixed seed), the transfer analysis predicts that its metrics *could* use a workload denominator and that `INVALID` would become rare. That is a testable prediction of the central claim, and the cleanest experiment available.
3. **Minimum experiment set.** One harness, three modes instrumented, probe suites published alongside the numbers.
4. **Probe-suite adequacy.** §9 names this as the sharpest hole. Any partial repair — calibration against a semi-recomputable harness (item 2) is the obvious candidate — is worth more than another metric.
5. **Run-identity gap.** Level-2 evidence needs a way to certify a transcript belongs to the run under audit. Unsolved; §6.3.
6. **Read the primary sources.** Appendix B is a verification log, not a literature review.

Appendix B — Verification log

B.1 Method

For each figure carried in an earlier internal draft under an `[REPORTED]` tag, a source was searched for. Status: **CONFIRMED** (source located, figure matches), **CORRECTED** (source located, figure differs and is now quoted accurately), **MISATTRIBUTED** (a source exists but measures a different quantity), **REMOVED** (no source could be established).

B.2 Results

Figure as originally carried	Status	What the source actually says
"65% of agent failures are confident false completion (2026 industry report)"	**CORRECTED**	Source located. The figure is **65.4%**, from a 2026 industry evaluation of Terminal-Bench-aligned tasks; the same project's failure-modes page puts it at **67.2%** of classified failing trials (2,574 trials). Both now quoted and distinguished.
"77.4% (a 2026 industry report)"	**MISATTRIBUTED** — removed, and worked as a case	No such false-completion rate located. 77.4 is a **Task Goal Completion** score for one method on an AppWorld leaderboard: a *success* rate, higher-is-better, close to the inverse. Worked in full at §8.1.
"`FCR` metric (EnterpriseBench, 2026)"	**CONFIRMED, with caveat**	`FCR` is defined in the repository README as the rate at which an agent claims success while the underlying state is wrong, and listed as implemented. The same status table lists the agent adapter and the first end-to-end experiment as **in progress**, and no `FCR` value is published. Cited with that caveat; name collision with two unrelated ACL benchmarks noted.
"`Zero-Bit Self-Audit` critique (2026)"	**REMOVED**	No work of that name located, in two searches. The related phrase "zero bits" does circulate in 2026 agent-discourse commentary — the argument that a witness sharing the author's substrate cannot emit an independent bit — but that is commentary, not a study, and it does not bear this name. Citing it as a critique would upgrade a blog post into a paper. Replaced with a locatable source: **no LLM-judge configuration tested exceeded AUROC 0.65**.
"agent self-assessment is a weak predictor of correctness"	**CONFIRMED**	Two sources, two quantities: the Terminal-Bench evaluation reports check-running at 76.4% of passing vs 76.8% of failing trials, and success assertions at 84.3% of failures vs 80.6% of passes; the benchmark study reports that no judge configuration exceeded AUROC 0.65. Different studies, different denominators — kept apart deliberately.

Figure as originally carried	Status	What the source actually says
(new) "four 2026 papers behind the term use different denominators"	**CONFIRMED**	The review says it in one sentence: the four papers "each measure a different quantity on a different denominator, so no headline number can be quoted against another". Used in §3.1, §8.1 and §8.2.

What this table rests on. Every row was re-checked against a retrieved copy of its source during final preparation: S1, S2, S4, S7 and S8 by fetching the page named in Appendix C, S3 by retrieving the paper, S5 and S6 through their ACL Anthology records, S9 through the dataset's paper abstract and card. The `Zero-Bit` row differs in kind — it records an **absence**, and an absence can be searched for but not verified; what is verified there is that no work of that name was found, not that none exists. No row in this table is `[CITED]` on the strength of internal consistency alone.

One source disagrees with itself: S3's abstract gives **45–48%** for single-control domains while its prevalence section gives **44–52%** for the same quantity. We quote the abstract and record the discrepancy rather than select the more convenient bound. A range quoted without the page it came from has the same defect this paper attributes to a percentage quoted without its denominator.

B.3 What remains unverified

Nothing in the body is tagged `[UNVERIFIED]`. Two caveats apply to the whole appendix:

1. **Near-primary, not primary.** Several confirmations rest on vendor pages, repository READMEs and secondary reviews rather than the full papers. Figures are quoted as those sources report them.
2. **Single access date.** All sources were read on 2026-11-01. Live pages change.

What is not external. All EquivSVA figures (80/120, 131, 79/93, 3 mutants) are from our own prior audit. **Our audit tooling for EquivSVA is a separate artifact and is not bundled with this paper**, so those four figures cannot be re-derived from anything in this repository — they are reported here as results of work not included, and should be read at that strength. All `[SEMANTICS]` blocks are different: they were executed for this paper, including the `PIPESTATUS` and locale comparisons in §5.2 and §7.3, and any reader can re-run them from the commands shown. `[OBSERVED]` items are single incidents by this author, n = 1.

B.4 What this appendix demonstrates

Three of five carried figures were wrong, and one was wrong in a way that inverted its meaning while keeping its appearance of authority. We did not find this by being suspicious in general. We found it by searching, which is the only operation that could have found it.

That is the paper's argument in its smallest form, and it applies to this appendix too: the absence of a `[REPORTED]` tag does not mean a figure was checked repeatedly. It means it was checked *once, on one date, at one depth.* Removing the tag replaced a claim about provenance with a claim about effort, and readers should price the two differently.

Appendix C — Sources consulted

Every `[CITED]` in the body resolves to a row below. Each row states **how deeply it was read**, because "a source exists" and "a source was read" are different claims and this paper's argument turns on the difference. Access date for all rows: **2026-11-01**.

#	Source	Used for	Depth
S1	Turing Benches, *Terminal-Bench 3.0 Failure Modes* — https://bench.turing.com/terminal-bench-3/failure/	67.2% of classified failing trials were confident false completion (2,574 trials); behaviour-class tallies	full page
S2	Turing, announcement of the above (LinkedIn, 2026-08-13) — https://www.linkedin.com/posts/turingcom_new-evals-from-turing-on-terminal-bench-30-activity-7493772720019759105-qQKx	65.4%; check-running 76.4% vs 76.8%; success assertions 84.3% vs 80.6%; the "derive a check from the same reading" formulation	full post text
S3	*From Confident Closing to Silent Failure: Characterizing False Success in LLM Agents* — https://arxiv.org/html/2606.09863	45–48% single-control, 3% dual-control, 75.8% self-assessing coding agents; no judge configuration above AUROC 0.65	full HTML
S4	`anote-ai` EnterpriseBench repository — https://github.com/anote-ai/research-enterprisebench	`FCR` definition; the implementation-status table lists the metric as implemented and the agent adapter plus first end-to-end experiment as **in progress**, with no value published	repository README
S5	EnterpriseBench (name collision), Findings of EMNLP 2026 — https://aclanthology.org/2026.findings-emnlp.932/	the unrelated benchmark sharing the name	abstract only
S6	*Can LLMs Help You at Work?* (a second, unrelated EnterpriseBench) — https://arxiv.org/html/2510.27287	the second name collision; rubric/pass-rate metrics rather than `FCR`	full HTML
S7	SOTA2 leaderboard, *Task and Scenario Goal Completion on AppWorld* — https://www.sota2.com/research/sota/task-and-scenario-goal-completion-on-appworld-normal-test	provenance of the 77.4 figure analysed in §8.1: it is a **Task Goal Completion** score, not a false-completion rate	leaderboard table
S8	LatentEval, *Silent failures: when agents report success and are wrong* — https://latenteval.ai/research/silent-failure-false-success	the observation that the four 2026 papers behind the term use different denominators, so no headline number can be quoted against another	full page

#	Source	Used for	Depth
S9	EquivSVA — *A Formally Verified Dataset of Behavioral Assertions Across Equivalent RTL Implementations*, https://arxiv.org/abs/2609.26751; repository https://github.com/aditigupta96/EquivSVA; dataset card https://huggingface.co/datasets/aditigupta/EquivSVA	the source corpus of §2.1: 120 behaviour families, four equivalent RTL implementations per family, three controlled mutants per family, a 17-job validation suite including reachability and mutant-distinguishability jobs	paper abstract + dataset card (not the full paper)

C.1 What is deliberately not listed

The `Zero-Bit Self-Audit` attribution is absent because no work of that name was found (Appendix B.2). This is an absence claim, and it is recorded as one: two searches located nothing under that name, which is not the same as establishing that nothing exists. The phrase "zero bits" does circulate in 2026 agent-discourse commentary — a witness sharing the author's substrate cannot emit an independent bit — but that is commentary without a citable study, and promoting a blog post into a citation is the failure this appendix is meant to prevent.

The four derived EquivSVA figures (80/120, 131, 79/93, 3 mutants) are absent as individual rows. They come from our audit *of* S9, and that audit's tooling is not bundled here (Appendix B.3). S9 is the source corpus; those four figures are our claims about it, not excerpts from it.

C.2 What "depth" means in the table above

"Full page" means the text was retrieved and read in its entirety. "Abstract only" means exactly that: the claim attributed to that row is limited to what an abstract establishes, which for S5 is the existence of a name collision and nothing more. "Repository README" means the implementation-status table, not a review of the code. "Paper abstract + dataset card" (S9) means the dataset's paper abstract and its Hugging Face card were read, but not the full paper and not the code: enough to confirm the corpus exists, its scale, and its validation-suite design — not enough to re-derive any of our four figures.

A reader who wants stronger evidence than this table provides should read S1–S3 in full and re-derive the percentages from the underlying trial counts. We have not done that, and Appendix A.6 records it as outstanding work rather than treating this table as its discharge.